



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр 806

Группа М8О-214СВ-24 Направление подготовки 02.03.02 «Фундаментальная информатика и информационные технологии»

Магистерская программа Информатика

Квалификация: магистр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА МАГИСТРА

На тему: Разработка и исследование пулов потоков с механизмом work-stealing на языке C++

Автор ВКРМ: _____ Спиридонов Кирилл Анатольевич (_____)

Научный руководитель: _____ (_____)

Консультант: _____ (_____)

Консультант: _____ (_____)

Рецензент: _____ (_____)

К защите допустить

Заведующий кафедрой № 806 _____ Крылов Сергей Сергеевич (_____)

_____ мая 2026 года

Москва 2026

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	3
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	4
1 Актуальность проблемы параллельного выполнения задач в современных программных системах	5
1.1 Тенденции развития многоядерных и многопроцессорных архитектур	5
1.2 Проблемы наивного подхода к управлению потоками	6
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	9

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе магистра применяют следующие термины с соответствующими определениями:

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе магистра применяют следующие сокращения и обозначения:

1 Актуальность проблемы параллельного выполнения задач в современных программных системах

1.1 Тенденции развития многоядерных и многопроцессорных архитектур

На протяжении нескольких десятилетий основным способом повышения производительности процессоров являлось увеличение тактовой частоты. Данная стратегия позволяла разработчикам программного обеспечения практически бесплатно получать прирост производительности при переходе на новое поколение оборудования. Однако в начале 2000-х годов этот подход столкнулся с фундаментальными физическими ограничениями: дальнейший рост тактовой частоты приводил к недопустимому увеличению тепловыделения и энергопотребления [1].

Ответом индустрии на данное ограничение стал переход к многоядерным архитектурам. Вместо наращивания производительности одного ядра производители начали размещать на одном кристалле несколько вычислительных ядер, каждое из которых работает на относительно невысокой тактовой частоте. Согласно данным компании Intel, уже к 2006 году большинство новых потребительских процессоров выпускалось с двумя и более ядрами, а к 2023 году флагманские серверные процессоры содержат сотни вычислительных ядер [2].

Данная тенденция нашла отражение в известном высказывании Херба Саттера, ставшем крылатой фразой в сообществе разработчиков: «Бесплатный обед закончился» [1]. Суть этого утверждения состоит в том, что простое обновление оборудования более не даёт автоматического прироста производительности для однопоточных программ. Вместо этого разработчики вынуждены явно проектировать свои приложения с расчётом на параллельное выполнение, чтобы задействовать все доступные вычислительные ресурсы.

Параллельно с развитием многоядерных процессоров активно развивались и многопроцессорные системы, в которых несколько физических процессоров объединяются в единую вычислительную платформу. Такие системы получили широкое распространение в области высокопроизводительных вычислений (HPC), центрах обработки данных и серверных инфраструктурах крупных интернет-компаний. Архитектура

NUMA (Non-Uniform Memory Access), характерная для многопроцессорных систем, вносит дополнительную сложность в разработку параллельных программ: время доступа к памяти зависит от того, на каком узле она физически расположена [3].

Таким образом, к середине 2010-х годов параллелизм превратился из узкоспециализированной темы, интересной лишь разработчикам суперкомпьютеров, в повседневную реальность для широкого круга инженеров-программистов. Веб-серверы, игровые движки, системы машинного обучения, базы данных и пользовательские приложения — все они вынуждены эффективно использовать параллелизм для достижения приемлемой производительности [4].

Рост числа ядер порождает принципиально новые требования к механизмам управления параллельным выполнением задач. Простые решения, приемлемые при двух-четырёх ядрах, демонстрируют неудовлетворительное масштабирование при 32, 64 и более ядрах. Именно поэтому исследование эффективных механизмов управления потоками и распределения задач между ними приобретает особую актуальность в современных условиях.

1.2 Проблемы наивного подхода к управлению потоками

Наиболее очевидным и простым подходом к параллельному выполнению задач является создание нового потока для каждой поступающей задачи и его уничтожение по завершении работы. Данный подход интуитивно понятен, прост в реализации и нередко используется в учебных примерах и прототипах. Однако при применении в производственных системах он обнаруживает ряд серьёзных недостатков.

Высокие накладные расходы на создание и уничтожение потоков. Создание потока операционной системой является дорогостоящей операцией. В операционной системе Linux создание потока через вызов `clone()` занимает порядка 5–15 мкс [5]. В операционной системе Windows аналогичная операция через `CreateThread()` имеет сопоставимую стоимость. При этом каждый поток требует выделения стека (как правило, от 64 КБ до 8 МБ по умолчанию), инициализации структур данных ядра и регистрации в планировщике операционной системы. Для высоконагруженных систем, обрабатывающих тысячи и десятки тысяч задач в

секунду, совокупные накладные расходы на управление жизненным циклом потоков могут составлять значительную долю общего времени выполнения [6].

Неконтролируемый рост числа потоков. В условиях пиковой нагрузки наивный подход приводит к лавинообразному росту числа одновременно существующих потоков. Поскольку каждый поток потребляет память (прежде всего под стек) и ресурсы ядра операционной системы, бесконтрольное создание потоков способно привести к исчерпанию памяти или достижению системного лимита на количество потоков. В Linux данный лимит определяется параметром ядра `/proc/sys/kernel/threads-max` и по умолчанию составляет порядка нескольких тысяч потоков. Превышение этого лимита влечёт отказ в создании новых потоков и, как следствие, отказ в обслуживании запросов [7].

Избыточное переключение контекста. Планировщик операционной системы распределяет процессорное время между всеми активными потоками посредством вытесняющей многозадачности. Переключение контекста — сохранение состояния текущего потока и восстановление состояния следующего — является затратной операцией, требующей нескольких микросекунд и приводящей к инвалидации кэшей процессора. Когда число активных потоков значительно превышает количество ядер, система тратит существенную долю времени на переключение контекстов вместо полезной работы [8]. Данный эффект получил название *thrashing* и является одной из ключевых проблем наивного подхода.

Отсутствие контроля над использованием ресурсов. Наивный подход не предоставляет механизмов ограничения количества одновременно выполняемых задач. В результате в периоды пиковой нагрузки система оказывается перегруженной, что негативно сказывается на латентности обработки всех запросов, включая приоритетные. Концепция *backpressure* — механизма обратного давления, при котором производитель задач замедляется при переполнении системы — принципиально несовместима с наивным подходом [9].

Неэффективное использование кэша процессора. Потоки, созданные для выполнения коротких задач, как правило, не имеют устойчивой привязки к конкретным ядрам процессора. Планировщик ОС может перемещать поток между ядрами при каждом пробуждении, что приводит к систематическим

промахам кэша и снижению эффективности работы с памятью. В современных процессорах разница в латентности между кэшем L1 и основной памятью достигает 200–300 раз, поэтому влияние данного фактора на производительность весьма существенно [10].

Перечисленные недостатки наивного подхода делают его непригодным для применения в производственных системах с высокими требованиями к производительности и предсказуемости. Это обстоятельство обусловило широкое распространение альтернативной концепции — *пула потоков*, при которой фиксированный набор заранее созданных потоков многократно используется для выполнения поступающих задач. Однако и в рамках концепции пула потоков остаётся ряд нерешённых проблем, связанных с балансировкой нагрузки между потоками, что будет рассмотрено в последующих разделах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Sutter H. The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software // Dr. Dobbs's Journal. — 2005. — Т. 30, № 3. — С. 202—210.
2. Intel Corporation. Intel® Xeon® Scalable Processors. — 2023. — URL: <https://www.intel.com/content/www/us/en/products/details/processors/xeon/scalable.html> (дата обращения 01.11.2024).
3. Lameter C. NUMA (Non-Uniform Memory Access): An Overview // ACM Queue. — 2013. — Т. 11, № 7. — С. 40—51. — DOI: 10.1145/2508834.2513149.
4. The Art of Multiprocessor Programming / М. Herlihy [и др.]. — 2-е изд. — Cambridge, MA : Morgan Kaufmann, 2020.
5. Drepper U., Molnar I. The Native POSIX Thread Library for Linux. — Red Hat Inc., 2003.
6. Butenhof D. R. Programming with POSIX Threads. — Boston, MA : Addison-Wesley, 1997.
7. Love R. Linux Kernel Development. — 3-е изд. — Boston, MA : Addison-Wesley, 2013.
8. Tanenbaum A. S., Bos H. Modern Operating Systems. — 4-е изд. — Hoboken, NJ : Pearson, 2014.
9. Reactive Streams Working Group. Reactive Streams Specification. — 2014. — URL: <https://www.reactive-streams.org> (дата обращения 01.11.2024).
10. Drepper U. What Every Programmer Should Know About Memory. — Red Hat Inc., 2007.